# Attack Report 1: Stealing the Password File

Oct 4, 2026 · Md Nasif Sarwar

This report shows how the @home server's static file handler lets an authenticated (or even unauthenticated) client read files outside the public/ folder, including the plaintext passwd file, and what an attacker does with it afterward.

## The vulnerability

In `httpd.js`, the function `serveStatic()` builds the file path like this:

```
const decodedPath = decodeURIComponent(url.parse(req.url).pathname);
const filePath = path.join(__dirname, 'public', decodedPath);
```

It takes whatever comes after the domain in the URL, decodes it, and glues it onto the `public/` folder with `path.join`. `path.join` does not stop `..` segments from walking back out of `public/` and into the rest of the project folder, including the `passwd` file sitting right next to `public/`, `templates/`, and `httpd.js` itself.

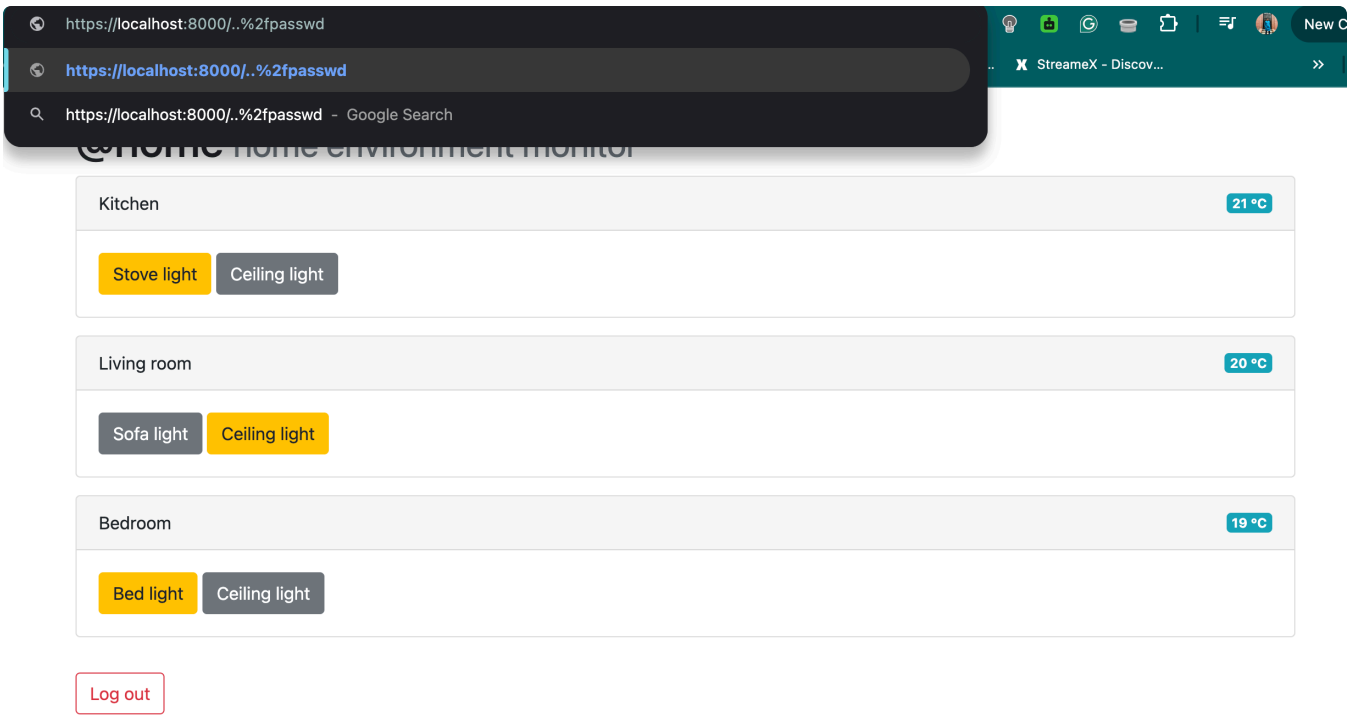
This matters for two reasons: `passwd` stores every username and password in plaintext (`{"daniel": "fisksoppa"}`), and `serveStatic()` is reached for *any* path that no other route matches - it is never protected by the login check, since the login page's own CSS/JS have to load before anyone is authenticated.

## Exploit 1: path traversal to steal passwd

First attempt, typing the traversal directly into the address bar as plain dots (`https://localhost:8000/public/../passwd`), failed with a 404. That is not the server rejecting the attack - it is the browser itself. Chrome normalizes `../` out of a URL before it ever sends the request, so the server never even saw a traversal attempt; it just got asked for `/passwd` directly, which does not exist as a route.

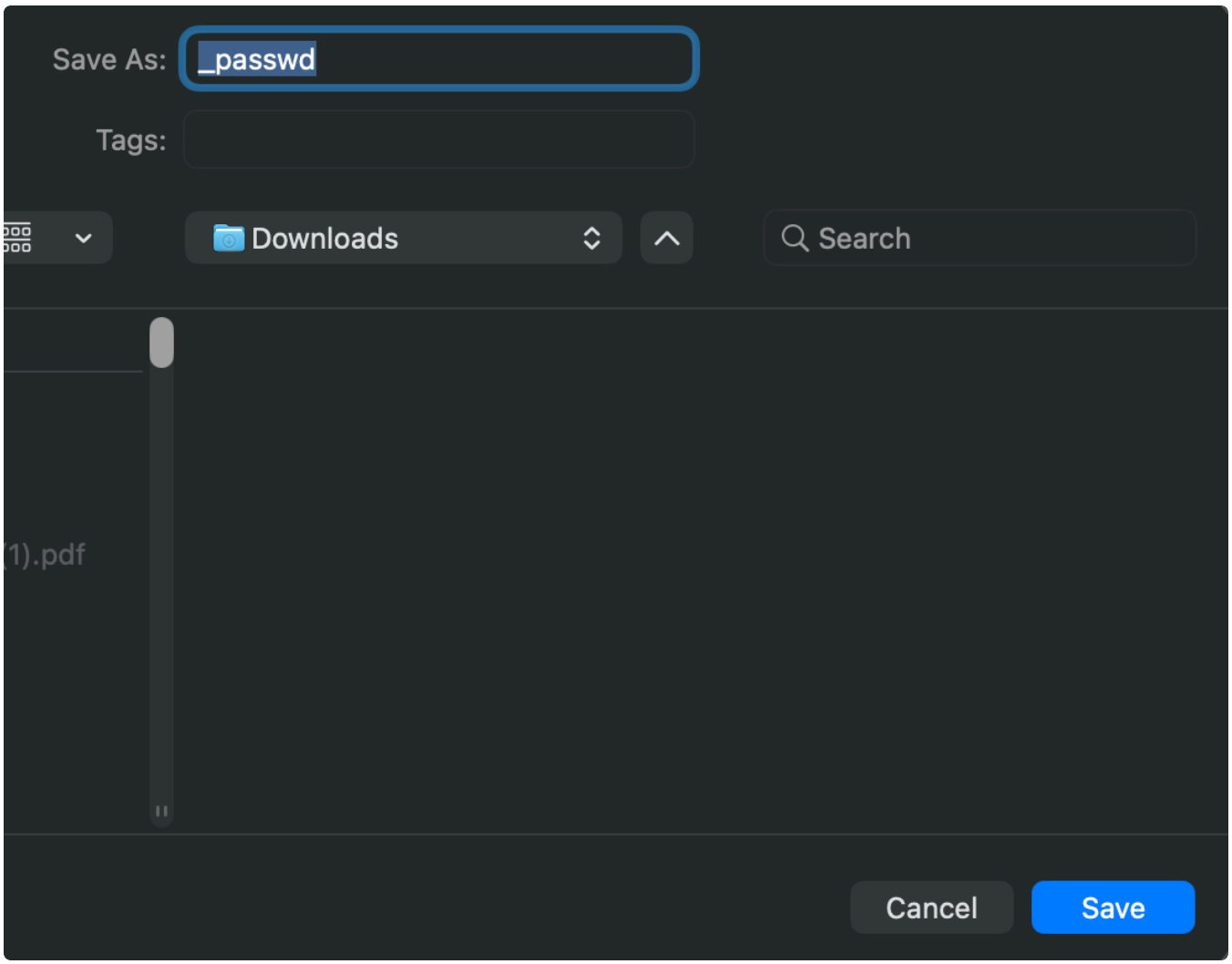
Working around that: URL-encoding the slash in `../` (`..%2f`) stops the browser from recognizing it as a dot-segment to clean up, so it reaches the server untouched:

```
https://localhost:8000/..%2fpasswd
```

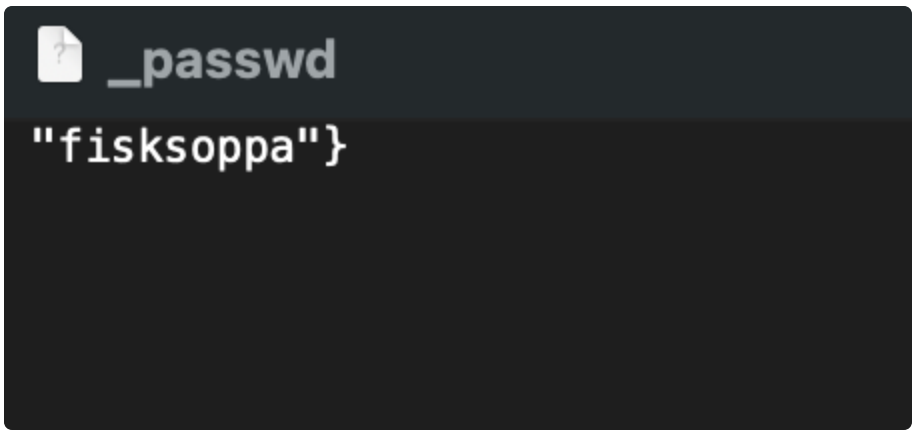



The server decodes `%2f` back to `/`, joins it onto the `public/` folder with `path.join(__dirname, 'public', decodedPath)`, and the `../` cancels the `public/` segment - landing right on the real `passwd` file, one level above where it should ever be reachable from.

The browser didn't render the JSON inline - it popped a download dialog instead:




That's a side effect of the server's MIME-type logic: `passwd` has no file extension, so `mimeTypes[ext]` finds nothing and falls back to `application/octet-stream`, which browsers treat as "download this, don't display it." The leak happens regardless - opening the downloaded file shows the stolen credentials in the clear:




```
{"daniel": "fisksoppa"}
```

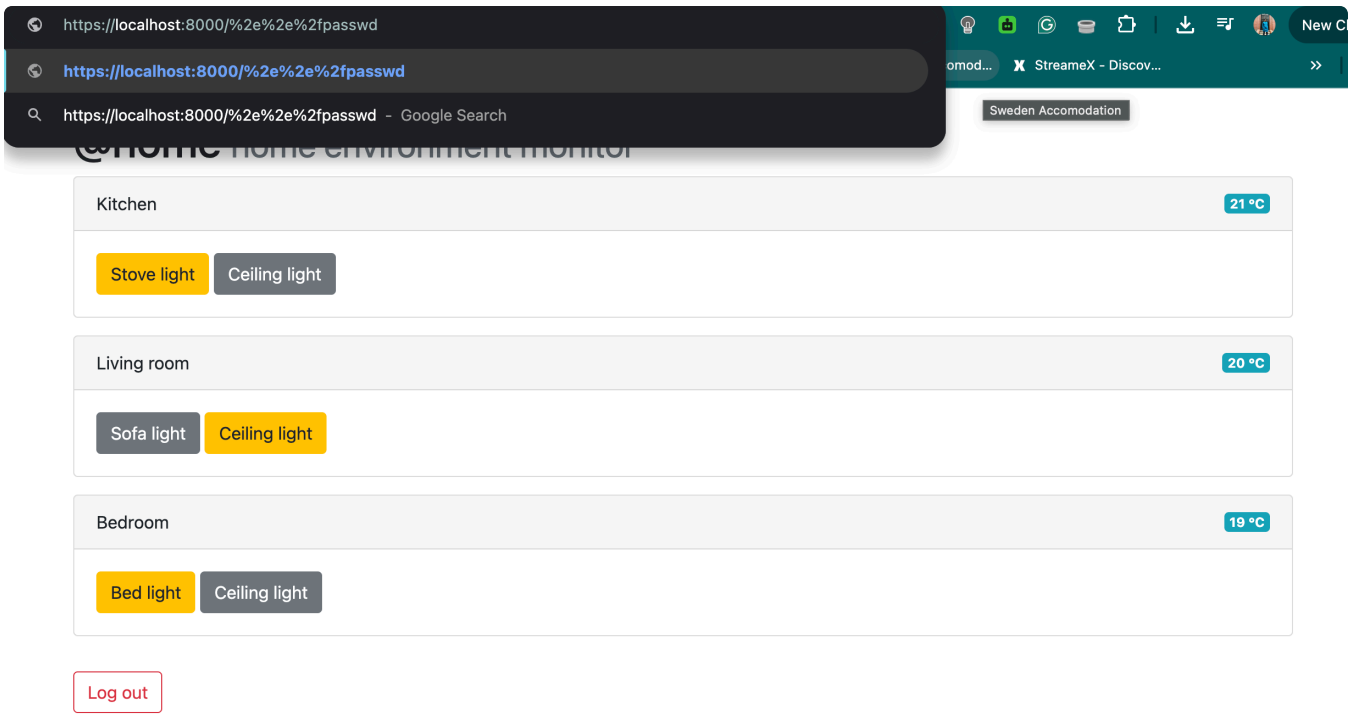
The entire credentials file - every username and plaintext password on the system - is now in the attacker's hands, without logging in, without guessing anything, using nothing but a single crafted URL.

## Exploit 2: fully-encoded traversal

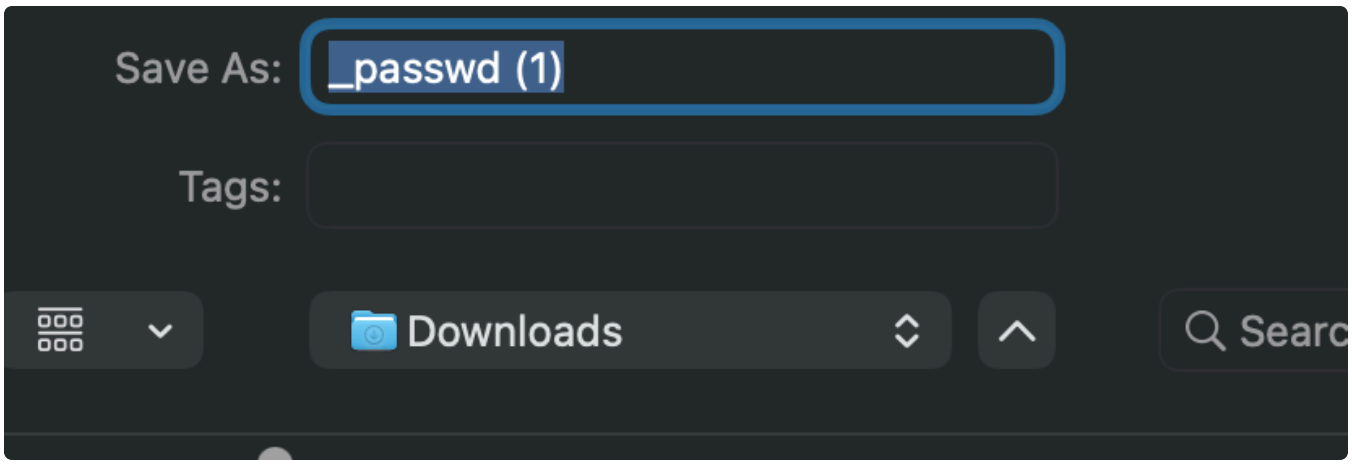
A narrower attempt first: encoding only the dots and leaving the slash literal (`https://localhost:8000/%2e%2e/passwd`) still 404'd. Here's why: `%2e` encodes `.`, which RFC 3986 classifies as an *unreserved* character, so Chrome freely decodes it back to `.` while still composing the request, rebuilds `..`, sees the literal `/` right after it, and normalizes the whole thing away before the request is even sent - same failure as the very first plain-dots attempt, just one layer deeper.

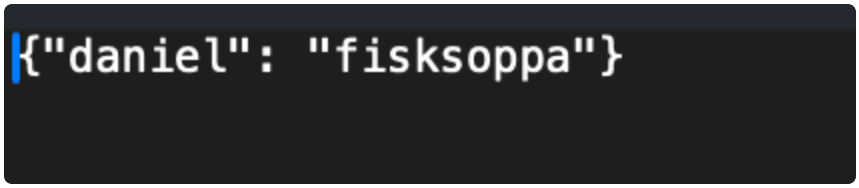
`%2f` encodes `/`, a *reserved* character, which browsers are required to leave alone. So encoding the slash as well as the dots removes every literal character the normalizer looks for:

```
https://localhost:8000/%2e%2e%2fpasswd
```




The address bar shows the URL exactly as typed, with no silent rewrite to `/passwd` this time. The server decodes it the same way as Exploit 1 and serves the same file - again as a forced download, again openable afterward:



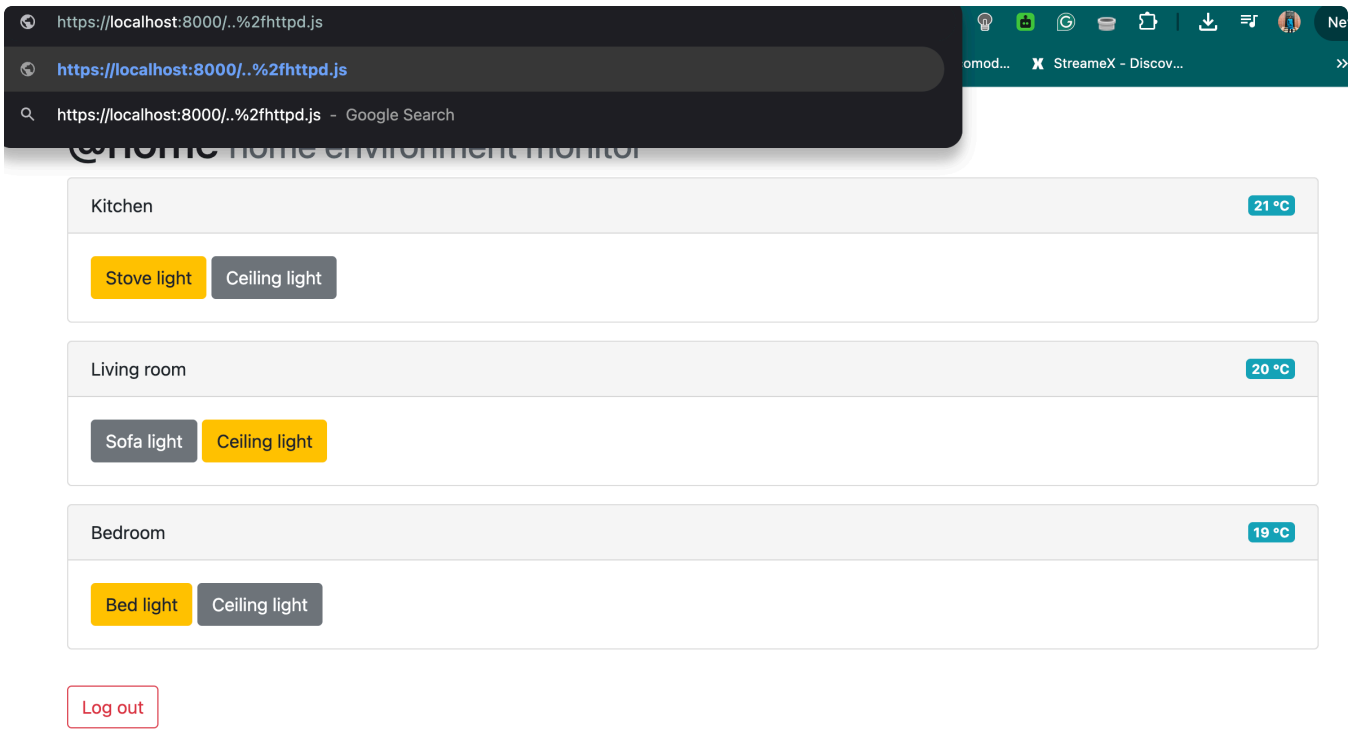

```
{"daniel": "fisksoppa"}
```

Two independently-encoded payloads, two successful leaks - confirming this isn't a one-off fluke but a structural hole in how `serveStatic()` resolves paths.

## Exploit 3: leaking server source code

The same hole isn't limited to `passwd` - anything sitting next to `public/` in the project folder is reachable, including the server's own source. Using the working encoding from Exploit 1:

```
https://localhost:8000/..%2fhttpd.js
```




This time the browser didn't prompt a download - `.js` has a mapped Content-Type (`text/javascript`), so it rendered straight in the tab, syntax-highlighted by the browser's own viewer:

```
// @home - mock home monitor app. HTTPS server, login with sessions,
// temperature + lights controlled over AJAX.

const https = require('https');
const fs = require('fs');
const path = require('path');
const url = require('url');

const PORT = 8000;

// cert and key for the self-signed HTTPS certificate
const httpsOptions = {
  key: fs.readFileSync(path.join(__dirname, 'cert', 'server.key')),
  cert: fs.readFileSync(path.join(__dirname, 'cert', 'server.crt')),
};

// fake sensor data, stands in for real devices
let data = {
  kitchen: {
    temperature: 21,
    lights: { stove: false, ceiling: false },
  },
  livingroom: {
    temperature: 20,
    lights: { sofa: false, ceiling: false },
  },
  bedroom: {
    temperature: 19,
    lights: { bed: false, ceiling: false },
  },
};
```

This is worse than the password leak on its own: it hands an attacker the entire server implementation - every route, the exact credential-check logic, how sessions are generated, the cookie name, all of it. Combined with Exploit 1/2′s leaked password, an attacker now has full read access to both *how* the login works and valid credentials to use against it - and, as Attack Report 2 shows, enough here to attack the session mechanism directly too.

## Offline password cracking

The lecture′s "full chain" for this attack is: traversal to steal the file, then an offline dictionary/brute-force attack against the stolen hashes, then account compromise. In this unpatched app there′s a shortcut in that chain: `passwd` stores passwords as plaintext (`{"daniel": "fisksoppa"}`), not as hashes. There is nothing to crack - the three exploits above already are the full compromise, no offline attack needed.

That′s worse than the chain describes, not better: even a weak unsalted hash would′ve forced some guessing. Plaintext skips that step entirely.

The offline-cracking step becomes meaningful once we add salted+peppered hashing in the Patched version (Report covers this once that patch lands): we'll take the same stolen-file scenario, point a small dictionary attack at a salted+peppered hash instead of a plaintext password, and measure how much harder it gets - directly demonstrating the salt/pepper slides from lecture rather than skipping past them.

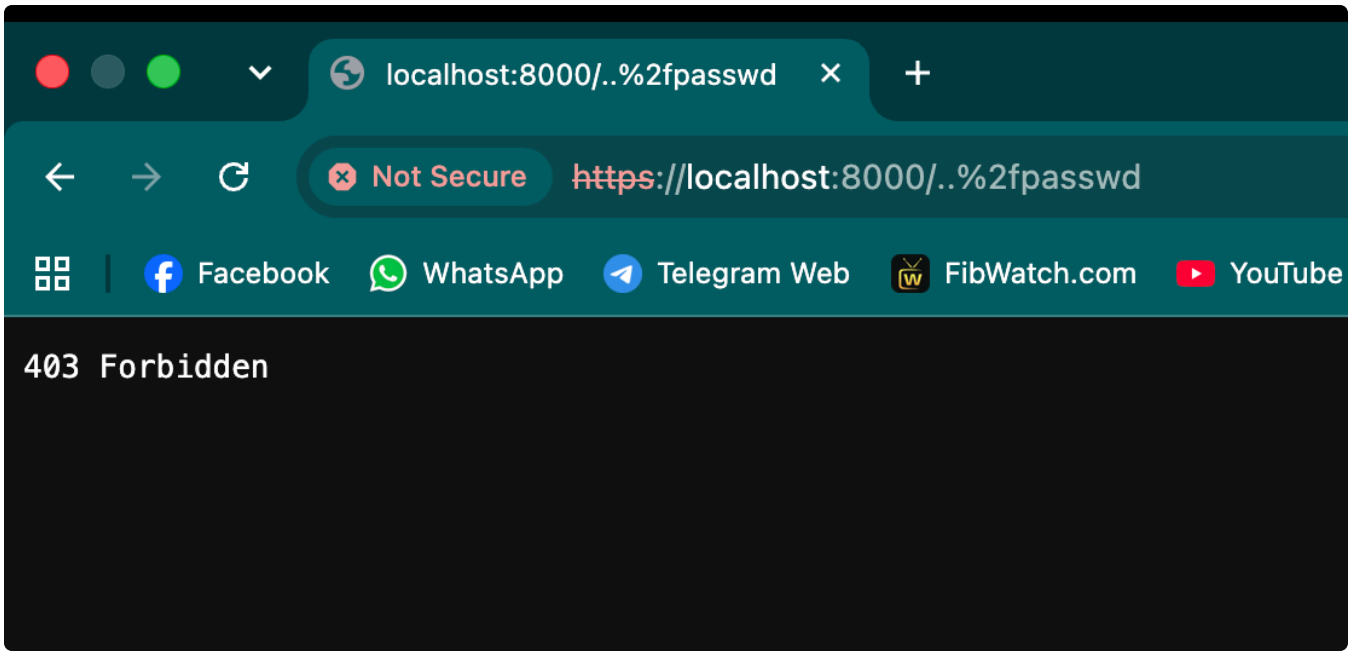
## Patch and re-test

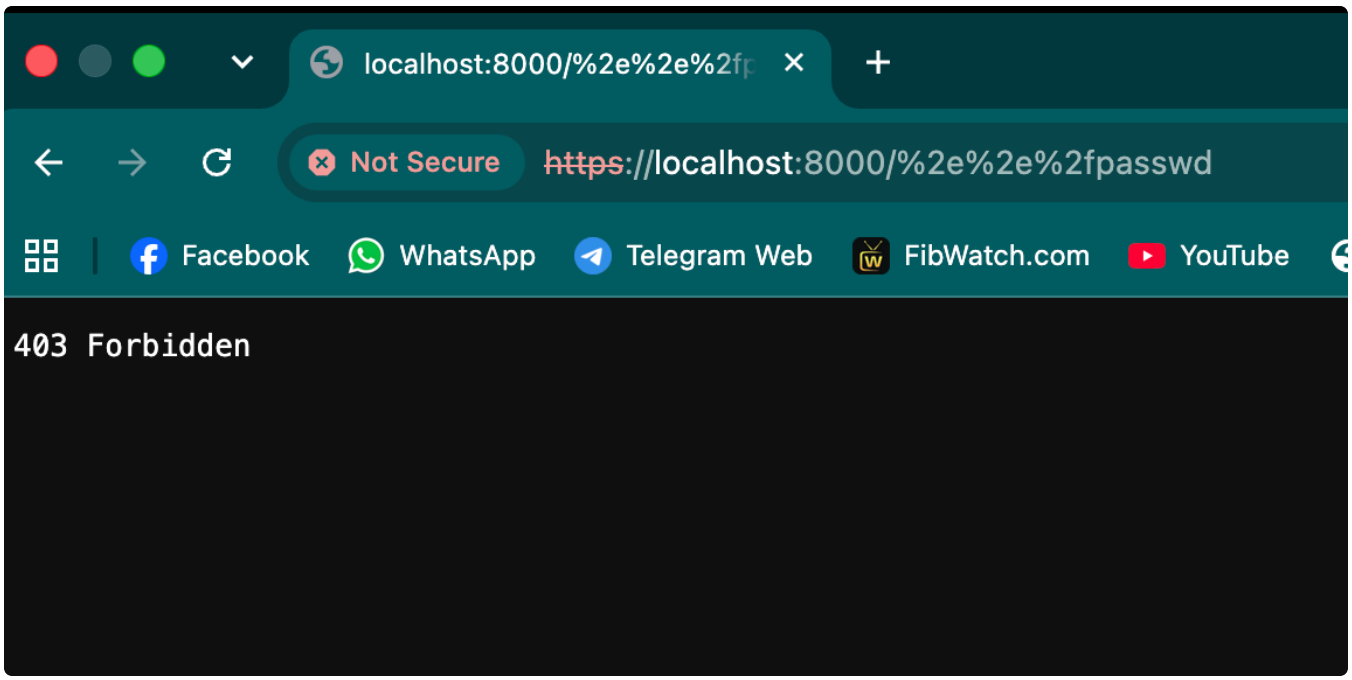
Fix, in `serveStatic()`: build the file path the same way, but resolve it and reject anything that lands outside `public/`.

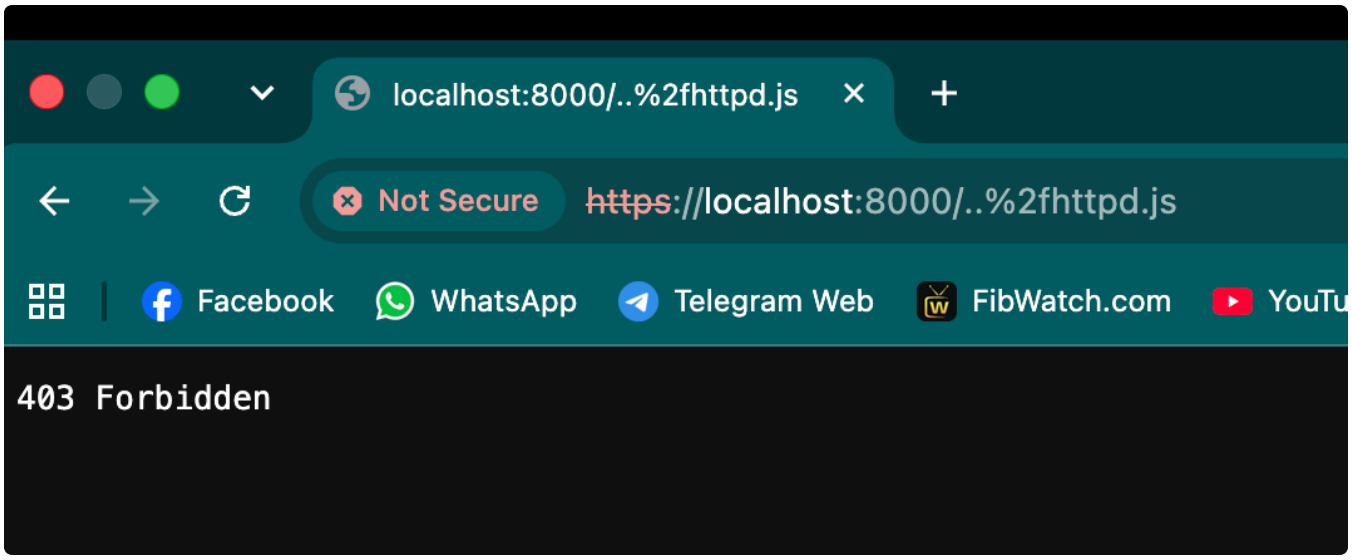
```
const publicDir = path.join(__dirname, 'public');
const filePath = path.join(publicDir, decodedPath);

// filePath must stay inside public/ - blocks ../ escaping the folder
if (filePath !== publicDir && !filePath.startsWith(publicDir + path.sep)) {
  res.writeHead(403, { 'Content-Type': 'text/plain' });
  res.end('403 Forbidden');
  return;
}
```

All three traversal payloads from above were re-tried against the patched server and now return 403, while normal static files (e.g. `/css/bootstrap.min.css`) still load fine.






### Defense in depth: hashing even if traversal still worked

The fix above already blocks all three traversal payloads. But `passwd` itself was also changed, so a future hole in `serveStatic()` wouldn't hand over the password again. Each user now gets a random salt, and the password is combined with that salt plus a server-side pepper (kept out of `passwd` entirely) before being run through `scrypt`, a slow hash:

```
const PEPPER = process.env.ATHOME_PEPPER || 'dva489-default-pepper';

function hashPassword(password, salt) {
  return crypto.scryptSync(password + PEPPER, salt, 64).toString('hex');
}

function checkPassword(username, password) {
  const record = credentials[username];
  if (!record) return false;
  const candidate = hashPassword(password, record.salt);
  const stored = Buffer.from(record.hash, 'hex');
  const attempt = Buffer.from(candidate, 'hex');
  return stored.length === attempt.length && crypto.timingSafeEqual(stored,
attempt);
}
```

`passwd` now looks like this instead of a plaintext password:

```
{
  "daniel": {
    "salt": "684cc4f2f323bf2110f1b4b9537598fa",
    "hash":
"81b7ead4dbfb4762224a5373105f5902ecaf7649565250a621bfe9d2de210bbf5f4b32f236e5
1cfba448bb36b3daf8bca288fa57bf9c2f827d24b04cfbd19c72"
  }
}
```

To check this actually holds up, the traversal guard was temporarily removed and the same `..%2fpasswd` exploit run again against this patched `passwd`. The file is still readable - traversal and hashing are two separate defenses - but this time it only hands over the salt and hash, never the password. A dictionary attack (8 common passwords, including the real one, `fisksoppa`) was then run against the stolen hash using only what the attacker actually has - the salt, not the pepper:

```
result: NOT FOUND - even the correct password fails without the pepper
time: 164 ms for 8 guesses
```

Even though `fisksoppa` was in the dictionary, it never matches, because the pepper is missing from the computation and the attacker has no way to get it from the stolen file. Running the same dictionary with the correct pepper (to confirm the hash was built correctly, not just untestable) does find it:

```
result: fisksoppa
time: 80 ms for 8 guesses (scrypt is deliberately slow)
```

So even in the worst case - traversal still working - the attacker walks away with a file that's useless without a second secret they don't have, and `scrypt` makes brute-forcing it expensive on top of that. The traversal guard was restored immediately after this test; the live Patched server has it active.